

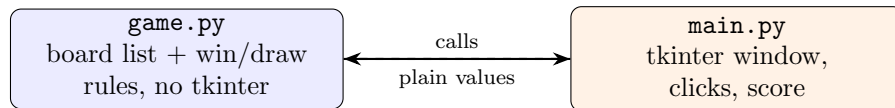
Tic Tac Toe (tkinter GUI) — Brief

Explainer document for the project's owner

1 What it is

A two-player Tic Tac Toe game with a real window, built on **tkinter** (Python's built-in GUI toolkit). Players click cells on a 3x3 button grid; the game detects wins across all 8 lines and draws, highlights the winning line, and keeps a running score across rounds. It replaces an earlier console-only version and fixes a real bug in that version (Section 3).

2 Architecture: two files, one job each

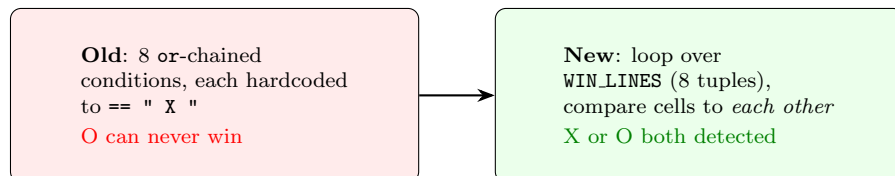


`game.py` is pure logic: a flat list of 9 cells (indices 0–8, left to right then top to bottom), each holding "X", "O", or "". It exposes `new_board`, `make_move`, `check_winner`, `is_draw`, and `is_game_over`. It has no `import tkinter` and no `print/input`, so its rules can be tested without opening a window.

`main.py` is the entire GUI: the window, the 3x3 grid of buttons, the status label, the “Play again” button, and the score label. It holds only display state (colors, the score dict) and calls into `game.py` for every rule decision — it never re-implements win or draw logic itself.

3 The original bug and the fix

The prior console version hardcoded 8 `or`-chained conditions, each comparing three cells to the literal string " X ". Because every condition tested for X specifically, an O win never satisfied any of them — the game simply never recognized O as a winner.



The fix, `check_winner`, loops over a list of the 8 winning index triples (`WIN_LINES`: 3 rows, 3 columns, 2 diagonals) and asks whether the three cells in a line are equal to *each other* and non-empty — not whether they equal a fixed literal. Whichever mark actually fills the line wins, so the same check works for both players with one code path instead of eight per-player branches.

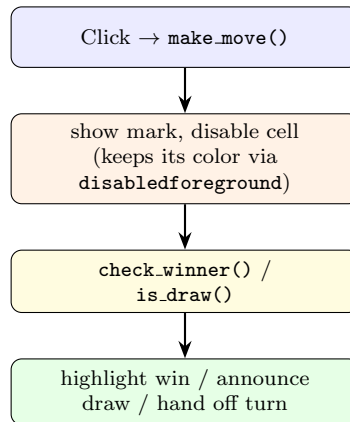
```
def check_winner(board):
```

```

for line in WIN_LINES:
    a, b, c = line
    if board[a] != EMPTY and board[a] == board[b] == board[c]:
        return board[a], line
return None

```

4 Click flow



A click calls `game.py`'s `make_move` (ignored if the game already ended or the cell is filled), then renders the mark and disables the button, then asks `game.py` whether that move won or drew the game, and finally either ends the round (highlighting the winning line, or announcing a draw) or passes the turn to the other player.

One notable display detail: tkinter grays out a disabled button's text by default, which would make X and O indistinguishable once placed. Setting `disabledforeground` explicitly keeps each mark's real color.

5 Score and reset

`reset.board` rebuilds the board and re-enables every button but deliberately does not touch the score dictionary, so X wins / O wins / draws accumulate across rounds until the app itself is restarted.

6 Glossary

- **tkinter**: Python's standard-library GUI toolkit.
- **Flat list**: a single one-dimensional list used to represent a 2D board via index arithmetic (`row * 3 + col`).
- **Tuple**: an immutable, ordered group of values.
- **Event loop** (`root.mainloop()`): the loop that waits for and dispatches user actions like clicks until the window closes.
- **Lambda**: an anonymous, inline function, used here to capture each button's own index for its click handler.